



06. 연산자 오버로딩

Heesung Oh

<https://github.com/3dapi>



6. 연산자 오버로딩과 사용자 정의 타입

- 사용자 정의 타입에 자연스러운 연산 제공
- 멤버 연산자와 비 멤버 연산자 구분
- 복합 대입 연산자와 산술 연산자
- 비교 연산자와 삼방향 비교
- 첨자 연산자
- 형 변환 연산자와 explicit
- 스트림 입출력 연산자
- 연산자 오버로딩의 설계 기준

사용자 정의 타입



타입의 의미에 맞는 연산 제공



기본형처럼 자연스럽게
사용하는 값 타입 구성



- 연산자 오버로딩은 문법을 짧게 만드는 기능이 아님
- 타입이 나타내는 개념에 맞는 연산 제공
- 코드가 문제 영역의 의미를 직접 표현하도록 구성
- 기본형의 연산자의 의미를 사용자 정의 타입으로 확장

```
int first = 10;  
int second = 20;  
int sum = first + second;  
bool same = first == second;
```

```
Vector3 result = first.Add(second);  
Vector3 result = first + second;
```

표현	의미
Add(first, second)	함수 이름으로 연산 설명
first + second	수학적 덧셈 의미를 직접 표현



- Vector3는 수학 벡터의 각 성분을 표현
- x, y, z를 직접 변경해도 타입의 유효성이 손상되지 않음
- 값 타입에서는 연산자 오버로딩이 특히 유용
- 자연스럽게 기대할 수 있는 연산은 연산자로 제공
- 여러 해석이 가능한 동작은 이름이 있는 함수가 적합

```
class Vector3
{
public:
    Vector3(float x, float y, float z)
        : x(x), y(y), z(z) {}
public:
    float x=0.0f, y=0.0f, z=0.0f;
};
```

```
Vector3 position = start + velocity * deltaTime;
Vector3 position = Add(start, Multiply(velocity, deltaTime));
```



6.1.2 기존 연산자 의미 유지

- 연산자의 의미를 크게 바꾸면 이해하기 어려움
→ 기본적인 개념을 유지

연산자	일반적인 의미	피연산자 변경
+, -, *, /	계산 결과를 새로운 값으로 생성	변경하지 않음
+=, -=, *=, /=	오른쪽 값을 현재 객체에 반영	왼쪽 객체 변경
==, !=	값의 동등성 비교	변경하지 않음
<, <=, >, >=	값의 순서 비교	변경하지 않음
[]	요소 접근	반환 참조에 따라 가능
++, --	현재 값을 한 단계 증가 또는 감소	현재 객체 변경
<<, >>	스트림 출력과 입력	스트림 또는 입력 대상 변경

```
Vector3 result = first + second;
first += second;
```

```
bool same = first == second;
std::cout << first << '\n';
```



- 하나 이상의 피연산자가 사용자 정의 타입
- 기본형 끼리 연산은 오버로딩 불가
- 새로운 연산자 기호 생성 불가 ← 오버로딩의 의미를 확인
- 피연산자 개수 변경 불가
- 우선순위, 결합 방향 변경 불가
 - ◆ 사용자 정의 타입에서도 *가 +보다 먼저 계산
- &&, || 오버로딩은 기본 논리 연산자의 단락 평가를 기대할 수 없음
 - ◆ 오버로딩된 논리 연산자는 일반 함수로 호출되어 모든 항이 사전 평가됨

```
// int operator+(int first, int second); // 허용되지 않음
```

```
Vector3 operator+(const Vector3& first, const Vector3& second);
```

```
Vector3 operator*(const Vector3& vector, float scalar);
```

```
Vector3 result = first + second * scalar;
```



- 형 변환 문법 자체는 오버로딩하지 않음
- 변환 생성자와 형 변환 연산자로 사용자 정의 변환 제공

오버로딩 불가	의미
::	범위 지정
.	직접 멤버 접근
.*	멤버 포인터를 이용한 직접 접근
?:	조건 연산자
sizeof	객체 또는 타입의 크기 확인
alignof	타입의 정렬 요구 사항 확인
typeid	런타임 타입 정보 확인
noexcept	표현식의 예외 발생 가능성 확인

멤버함수 구현	의미
=	대입
[]	인덱스 접근
()	함수 호출
->	포인터 형태의 멤버 접근





6.2 멤버 연산자와 비 멤버 연산자

- 연산자 함수는 멤버 함수 또는 비 멤버 함수로 정의 가능
- 구현 위치에 따라 피연산자 전달 방식이 달라짐
- 암시적 형 변환 적용 범위도 달라짐

기준	멤버 함수	비 멤버 함수
왼쪽 피연산자	현재 객체로 고정	매개변수로 전달
왼쪽 피연산자 암시적 변환	제한적	적용 가능
비공개 멤버 접근	직접 가능	friend, 공개 인터페이스 필요
대칭적인 이항 연산	덜 유연함	적합함
상태 변경 연산	적합함	멤버가 자연스러움

멤버 연산자
`first.operator+=(second)`

비 멤버 연산자
`operator+(first, second)`



6.2.1 멤버 함수로 구현하는 연산자

- 멤버 함수로 구현한 이항 연산자는 왼쪽 피연산자를 현재 객체로 사용
- 왼쪽 객체의 상태를 변경하는 연산에 자연스러움
- 대입, 복합 대입, 인덱스 접근은 멤버 함수가 적합

```
class Vector3
{
public:
    Vector3& operator+=(const Vector3& rhs)
    {
        x += rhs.x;
        y += rhs.y;
        z += rhs.z;
        return *this;
    }
public:
    float x=0.0f, y=0.0f, z=0.0f;
};
```

```
first += second;
first.operator+=(second);
```

- **first**는 현재 객체
- **second**는 매개변수로 전달



- 단항 연산자는 현재 객체 하나에 적용
- 별도의 피연산자 매개변수를 받지 않음
- 현재 객체 변경 없음 → 연산자는 상수 멤버 함수로

```
class Vector3
{
public:
    Vector3& operator+=(const Vector3& rhs)
    {
        x += rhs.x;
        y += rhs.y;
        z += rhs.z;
        return *this;
    }
public:
    float x=0.0f, y=0.0f, z=0.0f;
};

first += second;
first.operator+=(second);
```

연산자	성격	선언 예
<code>operator-()</code>	현재객체변경 X	<code>Vector3 operator-() const</code>
<code>operator+=()</code>	현재객체변경 O	<code>Vector3& operator+=(const Vector3&)</code>
<code>operator[]()</code>	원소 접근	<code>float& operator[](std::size_t)</code>



- 비 멤버 연산자 함수는 모든 피연산자를 매개변수로 받음
- 왼쪽과 오른쪽 피연산자를 대칭적으로 처리하기 쉬움
- 왼쪽 피연산자가 클래스 타입이 아닐 수 있는 연산에 필요

```
Vector3 operator+(Vector3 left, const Vector3& right)
{
    left += right;
    return left;
}
Vector3 result = first + second;
Vector3 result = operator+(first, second);
```

- left는 first의 복사본
- 복사본에 right를 더한 뒤 반환
- 원본 피연산자는 변경되지 않음

```
Vector3 operator*(Vector3 vector, float scalar);
Vector3 operator*(float scalar, Vector3 vector);
```

- vector * scalar와 scalar * vector는 별도 함수 필요



6.2.3 friend와 대칭적인 이항 연산자

- scalar * vector는 왼쪽 피연산자가 float
- Vector3의 멤버 함수로 구현할 수 없음
- 비 멤버 함수로 정의해야 함
- 공개 인터페이스만으로 구현 가능하면 friend가 필요하지 않음

```
Vector3 operator*(Vector3 vector, float scalar)
{
    vector *= scalar;
    return vector;
}
Vector3 operator*(float scalar, Vector3 vector)
{
    vector *= scalar;
    return vector;
}
```

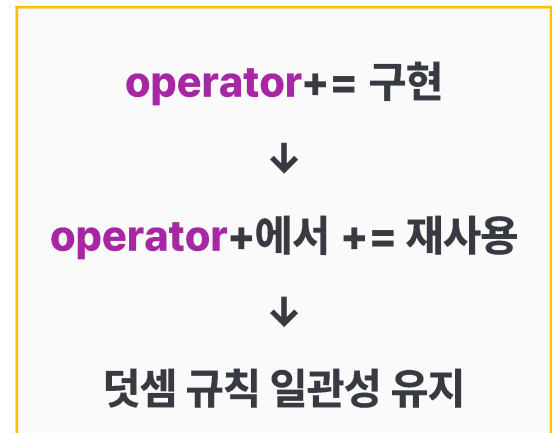
```
Vector3 first = value * 2.0f;
Vector3 second = 2.0f * value;
```

상황	선택
왼쪽 객체 직접 변경	멤버 함수
왼쪽 피연산자 비 클래스 타입	비 멤버 함수
비공개 멤버 직접 접근 필요	제한적 friend
공개 함수로 구현 가능	일반 비 멤버 함수



- 복합 대입 연산자를 먼저 구현
- 이항 산술 연산자가 복합 대입 연산자를 재사용
- 연산 규칙을 한곳에 모을 수 있음

연산자	역할	반환
+=	현재 객체에 더함	현재 객체의 참조
-=	현재 객체에서 뺌	현재 객체의 참조
*=	현재 객체에 스칼라 반영	현재 객체의 참조
+	새 값을 만들어 반환	새 객체
-	새 값을 만들어 반환	새 객체
*	새 값을 만들어 반환	새 객체





- 현재 객체를 변경
- 현재 객체의 참조 반환
- 기본형의 복합 대입 연산자와 같은 반환 규칙 제공

```
class Vector3
{
public:
    Vector3& operator+=(const Vector3& rhs)
    {
        x += rhs.x;
        y += rhs.y;
        z += rhs.z;
        return *this;
    }
    Vector3& operator*=(float scalar)
    {
        x *= scalar;
        y *= scalar;
        z *= scalar;
        return *this;
    }
public:
    float x=0.0f, y=0.0f, z=0.0f;
};
```

```
first += second;
first *= 2.0f;
```



6.3.1 상태 규칙이 있는 복합 대입

- 연산자라고 해서 클래스의 상태 규칙을 우회하면 안 됨
- 객체의 유효한 상태를 유지해야 함
- 값의 범위, 단위, 의미를 검증해야 할 수 있음

```
class Size
{
public:
    Size& operator--(int amount)
    {
        if (amount > value)
        {
            value = 0;
        }
        else
        {
            value -= amount;
        }
        return *this;
    }
private:
    int value = 0;
};
```

타입	검증이 필요한 예
분수	분모가 0이 되지 않아야 함
크기	음수가 되지 않아야 함
체력	0보다 작아지지 않아야 함
자원 수량	최대 보유량을 넘지 않아야 함



- 왼쪽 피연산자를 값으로 받아 복사본 생성
- 복사본에 복합 대입 연산 적용
- 복사본을 반환하여 원본 피연산자 유지

```
Vector3 operator+(Vector3 left, const Vector3& right)
{
    left += right;
    return left;
}
Vector3 operator-(Vector3 left, const Vector3& right)
{
    left -= right;
    return left;
}
Vector3 operator*(Vector3 vector, float scalar)
{
    vector *= scalar;
    return vector;
}
```

Vector3 result = first + second;
Vector3 scaled = value * 2.0f;

- 왼쪽 피연산자가 lvalue이면 복사 발생 가능
- 임시 객체이면 이동 활용 가능
- 반환 과정에서는 복사 생략이나 이동 적용 가능



- 한쪽 순서의 연산자를 정의했다고 반대 순서를 자동 생성하지 않음
- 교환 법칙이 성립하는 연산이라면 반대 순서 함수를 별도 제공
- 교환 법칙이 성립하지 않는 연산은 피연산자 순서를 바꾸면 안 됨

```
Vector3 operator*(float scalar, Vector3 vector)
{
    vector *= scalar;
    return vector;
}
```

```
Vector3 first = vector * 2.0f;
Vector3 second = 2.0f * vector;
Matrix result = first * second;
```

연산	순서 교환 가능성
벡터와 스칼라 곱	가능
행렬 곱셈	불가능
문자열 연결	순서 의미 있음
날짜와 기간 덧셈	설계에 따라 결정



- 단항 연산자는 하나의 피연산자에 적용
- 현재 객체를 변경하지 않는다면 상수 멤버 함수
- 새로운 값을 반환하는 형태가 자연스러움

```
class Vector3
{
public:
    Vector3 operator-() const
    {
        return Vector3(-x, -y, -z);
    }
    Vector3 operator+() const
    {
        return *this;
    }
public:
    float x=0.0f, y=0.0f, z=0.0f;
};
```

Vector3 reversed = -value;
Vector3 copied = +value;

- 단항 +의 의미가 필요하지 않다면 제공하지 않음
- 논리 부정 !은 상태의 반대 의미가 자연스러울 때만 사용
- 유효성 확인에는 explicit operator bool()이 더 자연스러울 수 있음



- 전위형과 후위형을 구분
- 전위형은 매개변수 없음
- 후위형은 구분용 int 매개변수 사용
- 후위형은 증가 전 값 반환, 복사 비용이 추가

```
class Counter
{
public:
    Counter& operator++()
    {
        ++value;
        return *this;
    }
    Counter operator++(int)
    {
        Counter previous = *this;
        ++(*this);
        return previous;
    }
public:
    int value = 0;
};
```

구분	선언	반환값
전위 증가	T& operator++()	증가된 현재 객체
후위 증가	T operator++(int)	증가하기 전 값 복사본
전위 감소	T& operator--()	감소된 현재 객체
후위 감소	T operator--(int)	감소하기 전 값 복사본



- 객체의 값 동일성과 순서를 표현
- 어떤 상태를 비교에 포함할지는 타입의 의미에 따라 결정
- 비교 과정에서 객체를 변경하지 않아야 함

```
struct Point
{
    int x, y;

    bool operator==(const Point& rhs) const
    {
        return x == rhs.x && y == rhs.y;
    }
};
```

```
Point first{10, 20};
Point second{10, 20};
```

```
bool same = first == second;
```

비교 기준	적합한 표현
값의 동등성	<code>operator==</code>
여러 동등성 기준	이름이 있는 함수
자연스러운 순서	<code><</code> , <code><=</code>
기준이 불명확한 순서	별도 비교 함수 객체



- ==는 객체의 메모리 표현 비교가 아님
- 객체가 나타내는 의미가 같은지를 비교
- 부동소수점 값은 정확한 동등 비교 부적합 가능
- 근사 비교는 이름이 있는 함수가 명확

```
bool Vector3::NearlyEquals(const Vector3& rhs, float epsilon) const
{
    return std::fabs(x - rhs.x) <= epsilon
        && std::fabs(y - rhs.y) <= epsilon
        && std::fabs(z - rhs.z) <= epsilon;
}
bool exact = first == second;
bool nearly = first.NearlyEquals(second, 0.0001f);
```

비교	의미
operator==	정확한 값 비교
NearlyEquals()	허용 오차 기반 근사 비교



- 값 사이에 자연스러운 순서가 있으면 순서 비교 제공
- 버전 정보는 주 버전, 부 버전, 패치 버전을 차례로 비교 가능
- 비교 함수는 일관된 순서를 제공해야 함

```
struct Version
{
    int major = 0, minor = 0, patch = 0;
    bool operator<(const Version& rhs) const
    {
        if (major != rhs.major)
        {
            return major < rhs.major;
        }
        if (minor != rhs.minor)
        {
            return minor < rhs.minor;
        }
        return patch < rhs.patch;
    }
};
```

```
Version first{1, 4, 2};
Version second{1, 5, 0};

bool older = first < second;
```



- 객체가 자기 자신보다 작다고 판단하지 않음
- $a < b$ 와 $b < a$ 가 동시에 참이 되지 않음
- $a < b$ 이고 $b < c$ 이면 $a < c$ 가 성립
- 동등한 값에 대해 비교 결과가 일관됨
- 여러 기준에서는 "<" 대신 이름 있는 비교 기준, 함수 객체가 명확

```
struct CompareVectorByLength
{
    bool operator()(const Vector3& first, const Vector3& second) const;
};
```

타입	순서 비교 제공 여부
버전	자연스러운 순서가 있음
날짜	자연스러운 순서가 있음
좌표	기준에 따라 달라짐
3차원 벡터	길이, x좌표, 사전식 등 기준이 여러 개



- C++20의 <=>는 두 값을 한 번 비교하여 작음, 같음, 큼을 표현
- 컴파일러는 <=>를 이용해 <, <=, >, >= 표현을 처리 가능
- 비교 결과는 0과 비교 가능
- 부동소수점형은 NaN 때문에 부분 순서를 가짐

```
#include <compare>
struct Version {
    int major = 0, minor = 0, patch = 0;
    std::strong_ordering operator<=> (const Version& rhs) const
    {
        if (major != rhs.major)
        {
            return major <=> rhs.major;
        }
        if (minor != rhs.minor)
        {
            return minor <=> rhs.minor;
        }
        return patch <=> rhs.patch;
    }
    bool operator==(const Version& rhs) const = default;
};
```




비교 범주	의미	대표 사례
<code>std::strong_ordering</code>	동등성과 순서가 완전하게 일치	정수, 버전 번호
<code>std::weak_ordering</code>	순서는 있지만 서로 다른 값이 동등한 순서로 취급될 수 있음	대소문자 무시 문자열 비교
<code>std::partial_ordering</code>	비교할 수 없는 값이 존재할 수 있음	NaN을 포함하는 부동소수점 비교

```
Version first{1, 4, 2};  
Version second{1, 5, 0};  
auto result = first <=> second;  
if (result < 0)  
{  
    std::cout << "first가 이전 버전\n";  
}
```

```
bool older = first < second;  
bool newer = first > second;
```



6.4.4 기본 비교 연산자 생성

- 모든 멤버를 선언 순서대로 비교해도 타입 의미와 일치하면 기본 생성 가능
- 반복되는 비교 코드를 줄일 수 있음
- 값 의미에 포함하지 않아야 하는 멤버가 있으면 직접 구현 필요

```
#include <compare>
struct Version
{
    int major = 0, minor = 0, patch = 0;
    auto operator<=>(const Version&) const = default;
};
```

major → minor → patch

```
struct Point
{
    int x, y;
    bool operator==(const Point&) const = default;
};
```

기본 비교가 적합한 경우

모든 멤버가 값 의미에 포함

선언 순서가 비교 순서와 일치

직접 구현이 필요한 경우

캐시, 통계, 디버깅 번호 제외 필요

비교 기준이 여러 개



- [] 오버로딩 → 사용자 정의 타입의 요소에 배열처럼 접근 가능
- 실제 요소의 참조 반환
- 일반 객체와 상수 객체의 접근 권한을 구분하려면 const 오버로딩 사용

class Vector3

```
{  
public:  
    float& operator[](std::size_t index)  
    {  
        return values[index];  
    }  
    const float& operator[](std::size_t index) const  
    {  
        return values[index];  
    }  
    float values[3] = {};  
};
```

```
Vector3 vector;  
vector[0] = 10.0f;
```

```
const Vector3 fixed{10.0f, 20.0f, 30.0f};  
float x = fixed[0];
```

호출 객체	선택 함수	반환 타입
Vector3	일반 버전	float&
const Vector3	상수 버전	const float&



6.5.3 범위 검사와 At()

- `operator[]`는 빠른 요소 접근 제공
- 범위를 검사하지 않는 형태가 많음
- 범위를 검사하는 별도 함수는 `At()` 형태로 제공 가능

```
class Vector3
{
public:
    float& At(std::size_t index)
    {
        if (index >= 3) {
            throw std::out_of_range("Vector3 index");
        }
        return values[index];
    }
    const float& At(std::size_t index) const
    {
        if (index >= 3) {
            throw std::out_of_range("Vector3 index");
        }
        return values[index];
    }
public:
    float values[3] = {};
};
```

함수	범위 검사	일반적인 목적
<code>operator[]</code>	X	인덱스가 유효하다고 보장된 빠른 접근
<code>At()</code>	O	외부 입력이나 계산 결과 접근



- *, -> 오버로딩 가능
- 포인터처럼 동작하는 사용자 정의 타입에서 사용
- 대표 사례: 스마트 포인터, 반복자, 포인터 래퍼

연산자	의미	일반적인 반환
*	가리키는 대상 객체에 접근	T&
->	대상 객체의 멤버에 접근	T*

```
T& operator*() const;  
T* operator->() const;
```

```
(*player).Move(); // operator*  
player->Move();    // operator->  
player.operator->()->Move();
```





- 형 변환 연산자는 객체를 다른 타입의 값으로 변환
- 반환 타입을 별도로 작성 안함
- 변환할 타입이 함수 이름의 일부
- 암시적 변환은 타입 의미가 명확하고 정보 손실이 없을 때만

```
class Length
{
public:
    explicit Length(double meters)
        : meters(meters)
    {
    }
    operator double() const
    {
        return meters;
    }
private:
    double meters = 0.0;
};
```

```
Length length(3.5);
double meters = length;

void PrintMeters(double value);

PrintMeters(length);
```



- 암시적 변환 제한 → 형 변환 연산자에 explicit 지정
- 명시적 변환은 허용
- 정보 손실, 큰 비용, 특별한 의미가 있으면 explicit
- 파일 크기가 int 범위를 벗어날 수 있다면 암시적 변환은 위험



6.7.3 explicit operator bool

- 객체가 유효한 상태인지 확인할 때
- explicit 이어도 조건식에서 사용 가능
- 일반적인 정수 변환 제한

```
class ResourceHandle
{
public:
    explicit ResourceHandle(void* handle)
        : handle(handle)
    {
    }
    explicit operator bool() const noexcept
    {
        return handle != nullptr;
    }
}
```

```
ResourceHandle resource(GetResource());
if (resource)
{
    std::cout << "자원을 사용할 수 있음\n";
}
bool valid = static_cast<bool>(resource);
// int value = resource; // 오류
```

- if (!resource)는 operator bool()의 결과를 반전하여 처리 가능





6.7.4 변환 생성자와 형 변환 연산자

구분	선언 위치	변환 방향
변환 생성자	변환 결과 타입	다른 타입 → 현재 클래스
형 변환 연산자	변환 원본 타입	현재 클래스 → 다른 타입

```
class Length
{
public:
    explicit Length(double meters);
};
class Length
{
public:
    explicit operator double() const;
};
```

```
class Fahrenheit
{
public:
    Fahrenheit(const Celsius& value);
};
class Celsius
{
public:
    operator Fahrenheit() const;
};
```

- 변환 결과 타입이 입력값으로 객체를 생성 → 변환 생성자
- 원본 타입이 자신의 값을 다른 표현으로 제공 → 형 변환 연산자
- 같은 변환 경로를 양쪽에 모두 제공 → 함수 선택이 모호 → 하나의 변환 책임만 제공



6.7.4 명시적 변환 또는 이름 있는 함수

- 암시적 변환은 변환의 의미가 분명하고 안전할 때 사용
- 중요한 동작을 숨기지 않아야 함

```
int Tolnt() const;  
std::string ToString() const;  
Duration ToSeconds() const;  
std::optional<int> TryTolnt() const;  
  
class FileSize  
{  
public:  
    std::optional<int> TryTolnt() const;  
    long long Bytes() const;  
};
```

명시적 변환 또는
이름이 있는 함수를 사용

정보 손실이 발생함

단위나 의미가 달라짐

실행 비용이 큼

실패할 수 있음

여러 변환 기준이 존재함

- 실패 가능성이 있으면 결과를 분명하게 표현하는 함수가 적합
- 단위 변환은 함수 이름으로 의미를 드러내는 편이 안전



- 사용자 정의 타입에 <<, >>를 제공하면 표준 스트림과 함께 사용 가능
- 왼쪽 피연산자가 스트림이므로 비 멤버 함수로 구현
- 스트림 참조를 반환하면 입출력을 연결할 수 있음

```
std::cout << vector;  
std::cin >> vector;  
std::ostream& operator<<(std::ostream& output, const Vector3& vector);  
std::istream& operator>>(std::istream& input, Vector3& vector);
```

연산자	왼쪽 피연산자	오른쪽 피연산자	변경 대상
<<	std::ostream&	출력할 객체	스트림
>>	std::istream&	입력받을 객체	스트림, 입력 대상

- 출력 대상은 보통 **const 참조**
- 입력 대상은 값이 바뀌어야 하므로 **비상수 참조**



- 출력 연산자는 줄 바꿈을 포함하지 않음
- 호출자가 출력 위치와 줄 바꿈을 결정
- 공개 접근자로 구현할 수 있으면 friend가 필요하지 않음

```
std::ostream& operator<<(std::ostream& output, const Vector3& vector)
{
    output << '('
        << vector.GetX() << ", "
        << vector.GetY() << ", "
        << vector.GetZ() << ')';
    return output;
}
```

```
std::cout << "position: " << position << '\n';
std::cout << first << ' ' << second << '\n';
```

```
std::cout
    → "position: " 출력
    → position 출력
    → '\n'
```

- 스트림 참조 반환으로 연속 출력 가능



- 입력 연산자는 비 멤버 함수로 구현
- 입력 대상 객체를 비상수 참조로 받음
- 임시 변수에 먼저 저장하고 입력이 성공한 뒤 객체에 반영
- 일부 입력 실패 시 기존 객체 상태 유지 가능

```
std::istream& operator>>(std::istream& input, Vector3& vector)
{
    float x=0.0f, y=0.0f, z=0.0f;
    if (input >> x >> y >> z)
        vector = Vector3(x, y, z);
    return input;
}
```

```
Vector3 position;
std::cin >> position;
if (std::cin)
{
    std::cout << position << '\n';
}
```

- 입력 형식이 복잡하면 구분 문자 검사와 실패 상태 설정 필요



6.8.3 비 멤버 함수와 friend 사용

- 스트림 연산자는 비 멤버 함수
- 비공개 멤버에 직접 접근할 수 없음
- 공개 접근자 함수로 구현하면 friend 불필요
- 출력 형식이 내부 표현과 밀접, 비공개 멤버 직접 접근
→ 제한적 friend

```
class Vector3
{
public:
    float values[3] = {};
private:
    friend std::ostream& operator<<(std::ostream& output, const Vector3& vector);
    friend std::istream& operator>>(std::istream& input, Vector3& vector);
```

선택	기준
공개 접근자 사용	내부 표현 의존을 줄이고 싶음
friend 사용	비공개 표현에 직접 접근해야 함

- friend 함수는 멤버 함수가 아님
- 함수는 여전히 비 멤버이며 스트림을 첫 번째 매개변수로 받음



6.9 연산자 오버로딩의 설계 기준

- 구현할 수 있다는 이유만으로 모든 연산자를 제공하지 않음
- 타입의 의미와 호출자의 기대를 기준으로 선택
- 연산자만 보고도 동작을 예측할 수 있어야 함

```
Vector3 result = first + second;
```

- 두 벡터의 성분별 덧셈으로 새로운 벡터를 만든다고 예상 가능

```
Vector3 newer = version + 1;
```

- 주 버전, 부 버전, 패치 버전 중 무엇을 증가시키는지 불분명
- `NextMajor()`, `NextMinor()`, `NextPatch()`가 더 명확

```
Meters Meters distance;
```

```
Seconds Seconds time;
```

```
MetersPerSecond MetersPerSecond speed = distance / time;
```

- 단위를 포함한다면 올바른 물리적 연산만 제공



- 값을 계산하거나 비교하는 연산에서 피연산자를 몰래 변경하지 않음
- 연산 비용과 실패 가능성을 숨기지 않음
- 논리 연산자 &&, || 오버로딩은 일반적인 타입에서 피함

```
Vector3 result = first + second;
```

- first가 변경되면 기본형의 +와 다른 의미
- 상태 변경이 목적이면 += 사용

```
bool same = first == second;
```

- 비교 연산에서 파일 출력, 로그 기록, 통계 증가 같은 부작용은 피하는 편이 좋음

피해야 할 설계 예

- +에서 파일 읽기나 네트워크 요청 수행
- ==에서 대규모 데이터베이스 조회
- []에서 예상하지 못한 요소 생성
- 형 변환에서 큰 데이터 복사
- 출력 연산자에서 객체 상태 변경



6.9.3 일반 함수가 더 적합한 경우

- 동작을 여러 의미로 해석할 수 있으면 이름이 있는 함수 사용
- 연산자의 일반적인 기대와 맞지 않으면 이름이 있는 함수 사용
- 비용, 실패 가능성, 변환 기준을 숨기지 않음

```
vector.Normalize();  
vector.Length();  
vector.Dot(rhs);  
vector.Cross(rhs);  
vector.NearlyEquals(rhs, epsilon);  
  
float dot = first.Dot(second);  
Vector3 cross = first.Cross(second);
```

```
object.Save(path);  
std::string text = object.ToString();  
std::optional<int> value = object.TryToInt();
```

연산자 제공 여부 기준

타입에서 자연스럽게 기대되는 연산

기본형의 같은 연산자와 비슷한 의미

예상 가능한 피연산자의 변경 여부

자연스러운 반환 타입

숨겨지지 않는 연산 비용, 실패 가능성

일반 함수 이름보다 명확한 의미 전달



- 값 의미 가지는 사용자 정의 타입
- 산술 연산
- 단항 부호 반전
- 동등 비교와 근사 비교
- 인덱스 접근과 범위 검사
- 스트림 입출력

```
class Vector3
{
public:
    Vector3() = default;
    Vector3(float x, float y, float z)
        : x(x), y(y), z(z) {}

    Vector3& operator+=(const Vector3& rhs);
    Vector3& operator-=(const Vector3& rhs);
    Vector3& operator*=(float scalar);

    Vector3 operator+() const;
    Vector3 operator-() const;

    bool operator==(const Vector3& rhs) const;
    bool NearlyEquals(
        const Vector3& rhs,
        float epsilon = 0.0001f) const;

public:
    float x=0.0f, y=0.0f, z=0.0f;
};
```



- +, *는 새 값을 반환
- +=, *=는 현재 객체 변경

```
Vector3& Vector3::operator+=(const Vector3& rhs)
{
    x += rhs.x; y += rhs.y; z += rhs.z;
    return *this;
}
Vector3& Vector3::operator*=(float scalar)
{
    x *= scalar; y *= scalar; z *= scalar;
    return *this;
}
Vector3 operator+(Vector3 left, const Vector3& right)
{
    left += right;
    return left;
}
Vector3 operator*(Vector3 vector, float scalar)
{
    vector *= scalar;
    return vector;
}
Vector3 operator*(float scalar, Vector3 vector)
{
    vector *= scalar;
    return vector;
}
```





- operator==는 정확한 비교
- NearlyEquals()는 부동소수점 계산 결과를 허용 오차로 비교
- 서로 다른 비교 의미를 하나의 연산자에 섞지 않음

```
bool Vector3::operator==(const Vector3& rhs) const
{
    return x == rhs.x
        && y == rhs.y
        && z == rhs.z;
}

bool Vector3::NearlyEquals(const Vector3& rhs, float epsilon) const
{
    return std::fabs(x - rhs.x) <= epsilon
        && std::fabs(y - rhs.y) <= epsilon
        && std::fabs(z - rhs.z) <= epsilon;
}

std::ostream& operator<<(std::ostream& output, const Vector3& vector)
{
    output << '('
        << vector.x << ", "
        << vector.y << ", "
        << vector.z << ')';
    return output;
}
```



- 연산자 오버로딩은 클래스와 열거형에 기존 연산자 의미를 정의
- 새로운 연산자를 만들 수 없음
- 우선순위, 결합 방향, 피연산자 개수를 변경할 수 없음
- 하나 이상의 피연산자는 사용자 정의 타입이어야 함
- 기본형끼리의 연산을 다시 정의할 수 없음
- 멤버 연산자는 왼쪽 피연산자를 현재 객체로 사용
- 복합 대입, 인덱스와 상태 변경 연산은 멤버 함수가 자연스러움
- 비 멤버 연산자는 모든 피연산자를 매개변수로 받음
- 대칭적인 이항 연산과 왼쪽 피연산자의 형 변환을 지원하기 쉬움
- 비공개 멤버 접근이 필요할 때만 friend를 제한적으로 활용





- 복합 대입 연산자는 현재 객체를 변경하고 참조 반환
- 이항 산술 연산자는 왼쪽 값을 복사한 뒤 복합 대입 연산자를 적용하여 구현 가능
- 증가와 감소 연산자는 전위형과 후위형의 반환 의미를 구분
- 동등 비교는 객체가 나타내는 값의 의미를 기준으로 구현
- 자연스러운 순서가 있는 타입만 순서 비교 제공
- 삼방향 비교 연산자와 기본 비교 생성으로 비교 코드 단순화 가능
- 첨자 연산자는 요소의 참조를 반환하여 배열 같은 접근 제공
- 역참조 연산자 "*" 는 대상 객체 자체에 접근, ->는 객체 멤버에 직접 접근
- 일반 객체와 상수 객체는 const 오버로딩으로 접근 권한 구분
- 빠른 비검사 접근과 범위를 확인하는 At()을 구분 가능





- 형 변환 연산자는 객체를 다른 타입으로 변환
- 반환 타입을 별도로 작성하지 않음
- 예상하지 못한 암시적 변환을 막으려면 explicit 지정
- 객체의 유효성 조건식에서 확인 → explicit operator bool()
- 변환 생성자와 형 변환 연산자를 같은 변환 경로에 중복 제공 안함
- 스트림 연산자는 왼쪽 피연산자가 스트림이므로 비 멤버 함수로 구현
- 스트림 참조를 반환하면 입출력 연결 가능
- 입력은 임시 값에 먼저 저장하고 검증 후 객체에 반영
- 연산자 오버로딩은 기본 연산자의 의미와 일관성을 유지할 때 활용
- 동작이 특별하거나 여러 의미로 해석되면 이름이 있는 일반 함수가 더 적합

